

Foundation models for medicine & AI for science

A detailed learning path, study plan and open-source apprenticeship guide

SEE IT > CODE IT > ENGINEER IT > REPRODUCE IT > CONTRIBUTE IT

From forward passes and attention to pretraining, inference, post-training and biomedical research. Designed for a medical/AI researcher building practical foundation-model expertise.

Three learning sequences • Eight intensive weekly modules • A 16-week alternative • Full source resource catalog • Biomedical project routes

Prepared 30 September 2026 | Clickable edition

Study links: [Source Google Doc](#) | [Source ChatGPT conversation](#)

The resource catalog preserves the courses and projects named in your Google Doc. The available conversation confirms the preference for PDF and hyperlinks inside the sequences. Added schedules, exercises, checkpoints and effort estimates are planning recommendations, not official course requirements.

Links are clickable throughout. Opening in a new tab or window is controlled by the PDF viewer; it cannot be enforced reliably by a PDF.

1 / How to use this plan

Start with a realistic scope

Use Sequence A unless you already train models comfortably or have a specific host repository in mind. The eight-week plan is an intensive first pass through selected materials, not completion of every university course or mastery of foundation-model research.

Planning assumption: 15-20 focused hours per week, about 120-160 hours for the core path. With 7-10 hours weekly, use the 16-week version. Add 2-4 preparatory weeks if Python, tensor operations or basic calculus are unfamiliar. These are estimates; advance by the checkpoints, not by the calendar.

Prerequisite diagnostic

- Python: write functions, use classes, read a traceback, load a small dataset and run a script from a terminal.
- Math: explain the chain rule, matrix multiplication, log probabilities, an expectation and a train/validation/test split.
- PyTorch: create tensors, inspect shapes, backpropagate a scalar loss and update parameters.
- Research practice: identify a baseline, control one variable, record a seed and preserve an untouched test set.

A repeatable weekly rhythm

- Session 1, 3 hours: watch the selected material and redraw its central mechanism from memory.
- Session 2, 3 hours: implement the smallest working version and check intermediate values.
- Session 3, 3 hours: run a controlled comparison and save the outputs.
- Session 4, 3 hours: inspect errors, explain a failure and revisit only the needed lecture sections.
- Session 5, 3-8 hours: finish the deliverable, write an experiment note and attempt the readiness checkpoint.

Keep a small evidence portfolio

Maintain one folder per module: notes, runnable code, configuration, data provenance, metrics, figures and a short README. A convincing portfolio contains eight small reproducible artifacts rather than a long list of watched videos.

Document map

Sections 2-4 compare three sequences. Section 5 gives eight weekly modules. Section 6 gives the slower schedule. Sections 7-8 contain the complete course and project catalog. Sections 9-11 cover biomedical specialization, apprenticeship and self-assessment. Section 12 records source and verification notes.

2 / Sequence A - Visual learner / fundamentals first (recommended default)

- [1. 3Blue1Brown neural networks/backprop visual intuition.](#)
- [2. Karpathy micrograd: implement autograd and backprop.](#)
- [3. 3Blue1Brown attention.](#)
- [4. Karpathy Build GPT: implement self-attention and a transformer.](#)
- [5. Karpathy Reproduce GPT-2: turn the toy implementation into a real training stack.](#)
- [6. Stanford CS336 selected lectures + Assignments 1-2.](#)
- [7. CMU inference course: KV cache, prefix sharing, speculative decoding, serving.](#)
- [8. Nathan Lambert: SFT > preference data > DPO/RL/RLVR.](#)
- [9. Ai2 OLMO or Marin: reproduce one small real experiment.](#)
- [10. Join a biomedical/biology project and contribute an evaluation, dataset, reproduction, or experiment report.](#)

Study links: [CS336 recordings](#) | [Marin alternative](#)

Best fit

Choose this route if you want intuition before systems detail. Spend the first two weeks building forward/backward and attention competence. Then follow the weekly modules in order. Use makemore as remediation when tensor operations or language-model losses remain unclear.

Exit condition

Explain every tensor in a small transformer, compare cached and uncached inference, and reproduce one small experiment.

3 / Sequence B - Systems/inference first

[1. Karpathy Build GPT.](#)

[2. Stanford CS336 resource accounting + GPUs/kernels/parallelism.](#)

[3. CMU inference: prefill/decode, KV cache, batching, PagedAttention, speculative decoding.](#)

[4. Run a local/small vLLM or equivalent serving experiment and measure tokens/sec, time-to-first-token, and memory.](#)

[5. Return to CS336 pretraining/data/scaling.](#)

[6. OLMo-core / Marin distributed training code.](#)

[7. Lambert post-training.](#)

[8. Biology/medicine project.](#)

Study links: [CS336 inference video](#) | [Marin alternative](#)

Best fit

Choose this route if you can already train a small model and your near-term work concerns serving, latency, memory or inference-time reasoning. Reorder the modules as 2, 3, 4, 5, 6, 7, 8; take module 1 whenever backpropagation is a gap. Do not infer correctness from a speedup: first compare outputs under matched settings.

Exit condition

Produce a reproducible benchmark with matched quality, documented hardware, latency distributions and memory measurements.

4 / Sequence C - Biology/medicine apprenticeship first

1. Learn just enough transformer mechanics: 3Blue1Brown attention + Karpathy Build GPT.
2. Clone MarinFold, Helico, MarinDNA, or OLMo-core; run tests and one documented small example.
3. Pick one experiment/report/issue in a biomedical/scientific domain.
4. Reproduce before proposing a new idea.
5. Use CS336/CMU/Lambert lectures on-demand to understand each subsystem you touch.
6. Submit an experiment note, evaluation/data contribution, documentation improvement grounded in a real reproduction, or a narrowly scoped PR.
7. Graduate to a small oncology-focused extension after you understand the host project's evaluation norms.

Study links: [Build GPT](#) | [Helico](#) | [MarinDNA](#) | [OLMo-core](#) | [CMU inference](#) | [Lambert course](#)

Best fit

Choose this route if a concrete biology problem motivates your learning. Start with two weeks of transformer basics, then alternate a repository task with the relevant foundational module. Reserve four to eight weeks for one reproduction. Domain expertise helps most when paired with a runnable example and an explicit evaluation protocol.

Exit condition

Reproduce a host-project result or a clearly bounded component, then write a contribution proposal supported by evidence.

5.1 / Week 1 Forward pass, loss, backpropagation and optimization

Study links: [Neural-network visual series](#) | [micrograd video](#) | [Zero to Hero materials](#)

Objective

Understand how a computation graph turns a prediction error into parameter updates.

Study and build

- 1. Draw a scalar graph and compute its forward values and local derivatives by hand. Watch the matching video sections after attempting the exercise.
- 2. Implement a tiny automatic-differentiation engine or reconstruct micrograd while explaining each operation. Check gradients against finite differences on a smooth toy example.
- 3. Train a tiny network on a toy dataset. Compare two learning rates with the same initialization; record loss and describe divergence or slow learning.

Deliverable

A one-page forward/loss/backward/update diagram, a working small network and a gradient-check notebook.

Readiness checkpoint

Without notes, explain gradient accumulation, zeroing gradients, chain rule and why the optimizer changes parameters after backward.

Scope and recovery

If gradients are wrong, reduce to one scalar operation. If Python is the bottleneck, spend a preparatory week on functions, classes and arrays.

Suggested allocation: 4-6 h study, 7-10 h implementation/experiments, 4 h evaluation and writing. Repeat or split the module if the checkpoint is not met.

5.2 / Week 2 Attention and a tiny transformer

Study links: [Attention visualization](#) | [Build GPT video](#) | [makemore sequence](#)

Objective

Trace token IDs through embeddings, causal attention, residual blocks, logits and cross-entropy.

Study and build

- 1. Write down B, T, C and head dimensions. Implement single-head attention and inspect Q, K, V, score matrix, mask and softmax rows.
- 2. Extend to multiple heads and a small transformer block. Check that modifying future input tokens does not change earlier logits in evaluation mode.
- 3. Train a tiny character model and sample with two temperatures. Explain why plausible text alone does not establish generalization.

Deliverable

An annotated tensor-shape diagram, causal attention code and a short generation comparison.

Readiness checkpoint

Explain scaling by the square root of head dimension, causal masking, residual connections and the difference between logits and probabilities.

Scope and recovery

If tensor shapes remain confusing, complete makemore bigram and MLP lessons before adding more layers.

Suggested allocation: 4-6 h study, 7-10 h implementation/experiments, 4 h evaluation and writing. Repeat or split the module if the checkpoint is not met.

5.3 / Week 3 Pretraining as a reproducible experiment

Study links: [GPT-2 reproduction video](#) | [CS336 course and assignments](#) | [CS336 recordings](#)

Objective

Turn an educational implementation into a measured, restartable small training run.

Study and build

- 1. Study the training-loop and resource-accounting material. Record model size, context length, effective batch size, tokenizer, optimizer and dataset split.
- 2. Run a tiny baseline, then vary learning rate, batch size, context length or width one at a time. Keep the token budget fixed where the comparison requires it.
- 3. Save and reload a checkpoint. Record validation loss, training loss, tokens/second and peak memory; explain the difference between optimization progress and generalization.

Deliverable

A baseline plus two controlled comparisons, learning curves and a checkpoint-resume note.

Readiness checkpoint

Explain effective batch size, gradient accumulation, validation leakage and which experimental comparisons are confounded.

Scope and recovery

The GPT-2 video is an engineering reference. You do not need to reproduce a full 124M-parameter training budget.

Suggested allocation: 4-6 h study, 7-10 h implementation/experiments, 4 h evaluation and writing. Repeat or split the module if the checkpoint is not met.

5.4 / Week 4 GPU efficiency, kernels and parallelism

Study links: [CS336 systems material](#) | [Lecture recordings](#)

Objective

Connect memory and compute accounting to profiler measurements.

Study and build

- 1. Draw the memory budget: parameters, gradients, optimizer states, activations and temporary buffers. Distinguish storage precision from arithmetic settings.
- 2. Profile attention and the MLP. Compare a reference attention implementation with an optimized supported path after checking numerical agreement.
- 3. Study data, tensor and pipeline parallelism. Write how gradients and parameters move in each. Attempt a small distributed exercise only with suitable hardware.

Deliverable

A resource-accounting sheet and benchmark with hardware, shapes, dtype, warm-up, synchronization and measurement method.

Readiness checkpoint

Explain why lower precision, checkpointing and gradient accumulation solve different constraints, and why kernel speedups depend on shapes.

Scope and recovery

On CPU or unsupported hardware, finish correctness checks and accounting. Mark GPU/Triton experiments deferred rather than reporting them as completed.

Suggested allocation: 4-6 h study, 7-10 h implementation/experiments, 4 h evaluation and writing. Repeat or split the module if the checkpoint is not met.

5.5 / Week 5 Inference, KV caches and serving

Study links: [CS336 inference video](#) | [CMU inference course](#)

Objective

Understand the separate costs of prompt processing and repeated token generation.

Study and build

- 1. Implement naive autoregressive generation, then cached generation. Under deterministic settings, compare token outputs and per-step logits within a stated tolerance.
- 2. Benchmark a grid of prompt lengths and output lengths. Record prefill time, decode time, time to first token, output tokens/second and cache memory.
- 3. Study MHA versus MQA/GQA, prefix sharing, continuous batching, PagedAttention, quantization and speculative decoding. Choose one for an additional conceptual or small practical comparison.

Deliverable

A request-to-prefill-to-cache-to-decode diagram, a correctness comparison and latency/memory plots.

Readiness checkpoint

Explain why KV caching avoids repeated work but grows memory, and why single-request latency differs from aggregate throughput.

Scope and recovery

Keep model, hardware, decoding settings and input lengths fixed when comparing implementations. CPU timing is not a GPU serving benchmark.

Suggested allocation: 4-6 h study, 7-10 h implementation/experiments, 4 h evaluation and writing. Repeat or split the module if the checkpoint is not met.

5.6 / Week 6 Data, scaling and biomedical adaptation

Study links: [CS336 scaling/data assignments](#) | [Ai2 training documentation](#)

Objective

Design a clean experiment before increasing data or compute.

Study and build

- 1. Build a small data manifest with source, license, document ID, split and processing rules. Inspect duplicates and overlap before tokenization.
- 2. Draft the source roadmap mixture sweep: general:biomedical ratios 100:0, 95:5, 80:20 and 50:50. Hold total tokens and training settings fixed. Run only the affordable subset.
- 3. Define both biomedical and general-domain evaluations. Include an untouched test set and document uncertainty and likely benchmark contamination. Study scaling-law fitting on small data, with explicit limits on extrapolation.

Deliverable

A preregistered experiment sheet, data-quality report and baseline evaluation; optional small mixture results.

Readiness checkpoint

Explain the difference between continued pretraining and SFT, and why added biomedical text can improve some outcomes while worsening others.

Scope and recovery

Use public or appropriately licensed material. Do not treat public accessibility as permission to train. A pilot need not execute the full sweep.

Suggested allocation: 4-6 h study, 7-10 h implementation/experiments, 4 h evaluation and writing. Repeat or split the module if the checkpoint is not met.

5.7 / Week 7 SFT, preference optimization and RL

Study links: [Lambert course](#) | [RLHF book](#) | [CS336 alignment assignment](#) | [Tulu reference](#)

Objective

Connect post-training objectives to concrete data and failure modes.

Study and build

- 1. Study foundations, instruction tuning, reward modeling and rejection sampling. Build a small instruction dataset with documented provenance and a separate evaluation set.
- 2. Run a small supervised fine-tuning experiment if hardware permits. Compare the same prompts before and after; inspect facts, omissions, unsupported claims and format adherence.
- 3. Study DPO and policy gradients. Optionally compare a preference method after the SFT baseline is stable. For an RLVR exercise, use a genuinely verifiable toy task and inspect reward exploits.

Deliverable

A before/after evaluation table, annotated errors and a training/data provenance sheet.

Readiness checkpoint

Explain policy, reference model, reward, KL regularization and preference pairs; distinguish reward improvement from task improvement.

Scope and recovery

Clinical prose rarely has a fully reliable automatic verifier. Do not equate a model judge or preference score with demonstrated clinical correctness.

Suggested allocation: 4-6 h study, 7-10 h implementation/experiments, 4 h evaluation and writing. Repeat or split the module if the checkpoint is not met.

5.8 / Week 8 Reproduction and open-source entry

Study links: [Marin](#) | [OLMo-core](#) | [MarinFold](#) | [Helico](#) | [MarinDNA](#) | [AstaBench](#)

Objective

Use one host project to turn coursework into research practice.

Study and build

- 1. Choose one repository. Read its README, license, setup instructions and one complete issue or experiment discussion, including failed attempts.
- 2. Pin a commit and reproduce a documented small example or a bounded component. Save commands, config, environment, metrics and differences from the reported setup.
- 3. Write an experiment note: question, expected result, actual result, failure analysis and the next discriminating experiment. Prepare one narrowly scoped contribution.

Deliverable

A runnable reproduction package and a concise contribution proposal grounded in its results.

Readiness checkpoint

Another researcher can follow your instructions, find the same inputs and understand any mismatch without asking you to reconstruct the run.

Scope and recovery

When a full run is too expensive, reproduce data preprocessing, a metric, a unit-scale model behavior or an evaluation subset and state the scope honestly.

Suggested allocation: 4-6 h study, 7-10 h implementation/experiments, 4 h evaluation and writing. Repeat or split the module if the checkpoint is not met.

6 / A sustainable 16-week schedule

Use the same eight modules at roughly 7-10 hours per week. The first week of each pair develops the concept and implementation; the second produces evidence and the written deliverable. A 24-week version can spread each module over three weeks at roughly 5-7 hours weekly.

Weeks 1-2: Forward pass, loss, backpropagation and optimization

First week: selected lessons and the smallest runnable implementation. Second week: a one-page forward/loss/backward/update diagram, a working small network and a gradient-check notebook.

Weeks 3-4: Attention and a tiny transformer

First week: selected lessons and the smallest runnable implementation. Second week: an annotated tensor-shape diagram, causal attention code and a short generation comparison.

Weeks 5-6: Pretraining as a reproducible experiment

First week: selected lessons and the smallest runnable implementation. Second week: a baseline plus two controlled comparisons, learning curves and a checkpoint-resume note.

Weeks 7-8: GPU efficiency, kernels and parallelism

First week: selected lessons and the smallest runnable implementation. Second week: a resource-accounting sheet and benchmark with hardware, shapes, dtype, warm-up, synchronization and measurement method.

Weeks 9-10: Inference, KV caches and serving

First week: selected lessons and the smallest runnable implementation. Second week: a request-to-prefill-to-cache-to-decode diagram, a correctness comparison and latency/memory plots.

Weeks 11-12: Data, scaling and biomedical adaptation

First week: selected lessons and the smallest runnable implementation. Second week: a preregistered experiment sheet, data-quality report and baseline evaluation; optional small mixture results.

Weeks 13-14: SFT, preference optimization and RL

First week: selected lessons and the smallest runnable implementation. Second week: a before/after evaluation table, annotated errors and a training/data provenance sheet.

Weeks 15-16: Reproduction and open-source entry

First week: selected lessons and the smallest runnable implementation. Second week: a runnable reproduction package and a concise contribution proposal grounded in its results.

Keep the scope bounded

One primary course stream, one implementation and one experiment at a time. Defer optional Berkeley breadth and deeper RL until a real question requires them. If you miss a week, move the schedule rather than doubling the next workload.

7 / Complete course and learning-material catalog

Everything named in the source course stack is retained below. Core means selected material belongs in the main path; depth means consult it for a specific gap. Public availability of lessons does not imply enrollment, grading access, free compute or a certificate.

The shortest high-quality path is: 3Blue1Brown for visual intuition; Andrej Karpathy for from-scratch code; Stanford CS336 for full-stack LM engineering; CMU/Graham Neubig for inference algorithms; Nathan Lambert/Ai2 for post-training; Ai2 OLMo and Open Athena/Marin for real open-model laboratory practice.

Visual intuition: 3Blue1Brown

- Neural networks / gradient descent / backpropagation - use these to make forward pass, loss, gradients, and parameter updates visually concrete.
- Attention in Transformers, step by step - <https://www.youtube.com/watch?v=eMlx5fFNoYc>
- Goal: be able to draw token embeddings > Q/K/V > attention scores > softmax > weighted values > output without looking anything up.

From-scratch coding: Andrej Karpathy

- Neural Networks: Zero to Hero - the overarching series: <https://karpathy.ai/zero-to-hero.html>
- micrograd: build autograd/backprop and a neural net from scratch - <https://www.youtube.com/watch?v=VMj-3S1tku0>
- makemore series: progressively build character language models and learn embeddings, MLPs, normalization, and training behavior.
- Let's build GPT from scratch - <https://www.youtube.com/watch?v=kCc8FmEb1nY>
- Let's reproduce GPT-2 (124M) - <https://www.youtube.com/watch?v=l8pRSuU81PU>
- Learning objective: understand every tensor in a transformer forward pass, then understand the training loop, optimizer, mixed precision, gradient accumulation, FlashAttention, distributed training, data loading, validation, and checkpointing.

Foundation-model engineering spine: Stanford CS336

- Stanford CS336 - Language Modeling from Scratch - <https://cs336.stanford.edu/>
- Lecture 1: overview and tokenization.
- Lecture 2: PyTorch/einops, FLOPs, memory, arithmetic intensity.
- Lecture 3: architectures and hyperparameters.
- Lecture 4: attention alternatives and mixture-of-experts.
- Lecture 5: GPUs and TPUs.
- Lecture 6: kernels and Triton.
- Lectures 7-8: parallelism and distributed training.

- Inference lecture: prefill vs decode, KV cache, memory, batching, and serving.
- Scaling lectures: how model size, token budget, learning rate, and compute interact.
- Data lectures: Common Crawl, filtering, deduplication, decontamination, and dataset mixtures.
- Post-training/alignment lectures: supervised fine-tuning and reasoning-oriented RL.
- Assignment 1: implement tokenizer, transformer, optimizer, and train a minimal LM.
- Assignment 2: profile/benchmark, implement FlashAttention2 in Triton, and build memory-efficient distributed training.
- Assignment 3: fit scaling laws.
- Assignment 4: turn raw web data into usable pretraining data with filtering and deduplication.
- Assignment 5: SFT + RL for reasoning; optional DPO/safety-alignment work.

Inference and serving: CMU / Graham Neubig

- CMU 11-664/763 - Inference Algorithms for Language Modeling (Fall 2025) - <https://www.phontron.com/class/lminference-fall2025/>
- Treat this as the semester-long companion to CS336's inference lecture.
- Topics to prioritize: sampling; beam/A* search; reasoning models; best-of-N; inference-time scaling; prefix sharing; KV-cache optimization; speculative decoding; sparse attention; batching; serving libraries.
- Do the programming homeworks if possible; inference becomes much clearer when you implement the decoder rather than only watch slides.
- CMU 11-711 Advanced NLP 2024 - broader model-building companion - <https://www.phontron.com/class/anlp2024/>
- Assignment 1 from Advanced NLP: Build Your Own LLaMA. Later assignments include RAG, SOTA reimplementation, and a research project.

Post-training and RLHF: Nathan Lambert / Ai2

- Nathan Lambert's free RLHF & Post-Training Course - <https://rlhfbook.com/course>
- Free RLHF/Post-Training book - <https://rlhfbook.com/>
- Lecture 0: ML foundations refresher - LM probabilities, cross-entropy, KL, gradients, SFT.
- Lecture 1: RLHF/post-training overview.
- Lecture 2: instruction fine-tuning, reward models, rejection sampling.
- Lectures 3-4: RL motivation, math, implementation, loss aggregation, async training.
- Lecture 5: reasoning models and RLVR / inference-time scaling.
- Lecture 6: DPO and variants.
- Lecture 7: synthetic data, on-policy distillation, AI feedback, rubrics.
- Lectures 8-10: preference data, over-optimization/reward hacking, regularization/generalization.

- Use Lambert after you understand a basic transformer training loop; otherwise the optimization details float without an anchor.

Open model laboratory: Ai2 OLMo

- Ai2 developer docs - <https://docs.allenai.org/>
- Training models / OLMo 3 - https://docs.allenai.org/in_depth/training
- OLMo-core GitHub - <https://github.com/allenai/OLMo-core>
- Study the real stack: dataset/tokenization > model config > optimizer > distributed training > checkpointing > evaluation > post-training.
- Use OLMo recipes as a reference implementation after doing smaller educational code from Karpathy/CS336.

Berkeley courses worth adding

- UC Berkeley CS 194/294-267 - Understanding Large Language Models: Foundations and Safety - https://rdi.berkeley.edu/understanding_llms/s24
- Best for foundations, scaling, interpretability, reasoning, evaluation, robustness, privacy, unlearning, and safety.
- UC Berkeley Advanced Large Language Model Agents (Spring 2025) - <https://rdi.berkeley.edu/adv-llm-agents/sp25>
- Best for inference-time reasoning, post-training for reasoning, search/planning, tool use, code, mathematics, and agentic workflows.
- UC Berkeley CS285 / Deep Reinforcement Learning - use selected policy-gradient / actor-critic / RL lectures as background for post-training - <https://rail.eecs.berkeley.edu/deeprlcourse/>
- Don't do all of Berkeley before touching models. Use it as targeted depth when a topic appears in your experiments.

Additional navigation and current-course notes

Study links: [3Blue1Brown neural-network playlist](#) | [CS336 2026 recordings](#) | [CS336 lecture 10: inference](#)

The CS336 home page is the Spring 2026 offering. Use its topic headings and assignment handouts to match the source roadmap. The inference video is a Stanford CS336 lecture; it is not a CMU lecture.

Lambert now also lists lecture 11 on tools/agents, lecture 12 on evaluation and lecture 13 on character training. These are optional follow-ons after the source roadmap lectures 0-10. The same course hub provides recordings, slides and PDFs.

Study links: [Lambert current course](#)

Karpathy also lists a GPT tokenizer lesson. Use it alongside CS336 tokenization if text-to-token mechanics remain unclear. All five makemore lessons can be reached from the Zero to Hero syllabus.

Study links: [Full Karpathy syllabus](#)

8 / Open-model and AI-for-science laboratory

- Marin repository - <https://github.com/marin-community/marin>
- Open Athena - <https://openathena.ai/>
- Start with Marin's TinyStories first experiment, then the LM-training tutorial, then experiment reports and retrospectives.
- Read experiment reports as laboratory notebooks: data mixtures, optimizer choices, architecture changes, failed runs, loss spikes, scaling decisions, and efficiency work.
- Use mumwelt to search Marin's code/issues/PRs/experiments/Discord/W&B context - <https://github.com/Open-Athena/mumwelt>
- Core habit: for each design choice ask: what hypothesis led to it, what experiment tested it, what failed, what metric changed, and can I reproduce it at 1/1000 scale?

Open Athena AI-for-science / biology projects

- Open Athena projects page - <https://openathena.ai/projects/>
- BoltzGen: target-specific protein generation / binder design; relevant to antibodies, mini-proteins, and drug-discovery workflows.
- Marifold - <https://github.com/Open-Athena/Marifold> - open protein-structure/modeling experiments and an especially attractive contributor entry point.
- Helico - <https://github.com/Open-Athena/helico> - AF3-like architecture/training experimentation with preprocessing, single/multi-GPU training, W&B, and benchmarking.
- MarifoldDNA - open genomic-language-model work; useful if you want to connect foundation-model engineering with variant-effect prediction and functional genomics.
- PlantCAD / GeneCAD: DNA foundation models for plant genomics; useful for seeing how domain data changes the modeling problem.
- For oncology, no single Open Athena project should be assumed to be a dedicated oncology model; the closest transferable routes are genomic variant modeling, binder/protein design, structure prediction, and biomedical/scientific evaluation.

Ai2 AI-for-science routes

- OLMo / Tülu: fully open language-model and post-training pipelines.
- Asta / AstaBench: agents and evaluation for scientific research workflows such as literature research, coding/data analysis, and hypothesis-oriented tasks.
- CellOLMo-style biology direction: multimodal grounding of language models in cellular/molecular data is a particularly relevant conceptual template for future tumor single-cell and translational-oncology work.
- For a medical-AI apprenticeship, do not wait for a project literally labeled 'oncology'. Join a technically adjacent open project where your biomedical evaluation/data expertise is useful.

Direct entry links added for named projects

Study links: [MarinDNA repository](#) | [Tulu overview](#) | [AstaBench overview](#) | [AstaBench code](#)

Verification limits

MarinFold, Helico and MarinDNA have identifiable repositories; inspect their current setup, data access and compute requirements before choosing a task. Open Athena lists BoltzGen and PlantCAD/GeneCAD on its projects page. Their relevance to oncology is a possible transfer direction, not evidence of clinical utility.

The source mentions a "CellOLMo-style" direction. A corresponding official Ai2 project was not verified in this check. Retain it as a conceptual interest in cellular/molecular grounding, not as a confirmed course, released model or apprenticeship target.

9 / Medicine and oncology specialization

- Continued pretraining: compare small amounts of high-quality biomedical text with indiscriminate large domain mixtures.
- Clinical post-training: focus on tasks with clear provenance and evaluation, not vague 'medical intelligence'.
- Evaluation dimensions: factuality, clinical relevance, omission rate, hallucination/unsupported-claim rate, terminology accuracy, structured-field accuracy, calibration, subgroup performance, and clinician review where appropriate.
- Contamination discipline: keep benchmark questions and close paraphrases out of training data.
- Privacy: use de-identified/public/licensed data; treat PHI and hospital data as a separate governance/security problem.
- Oncology project ideas after reproduction competence: somatic variant-effect prediction; tumor single-cell representation + language; oncology literature agents; trial eligibility extraction; radiotherapy workflow copilots; cancer-biology binder design; longitudinal clinical reasoning evaluation.

Choose one tractable first project

Literature and trial eligibility

Start with a small public, license-compatible corpus and an explicit extraction schema. Compare prompting or retrieval against SFT only if a baseline shows a training need. Measure per-field precision/recall, omissions and evidence attribution. Keep document-level separation between train and test.

Genomics and variant modeling

Use MarinDNA as an engineering entry point. First reproduce its native task and metrics. A later cancer-relevant experiment should define the assay/label, split strategy and comparison baseline; avoid treating a representation score as validated pathogenicity.

Protein structure and binder design

Use MarinFold, Helico or the BoltzGen project entry point. Begin with input preparation, evaluation or a small documented inference/reproduction task. Computational structure or binding scores do not establish biological efficacy; experimental validation is a separate stage.

Scientific agents and virtual-patient evaluation

Use AstaBench for scientific workflow evaluation, or define a small offline simulated task. Score tool-call validity, evidence use, task completion, omissions and failure recovery. Virtual-patient performance does not establish performance with real patients.

Study links: [MarinDNA](#) | [MarinFold](#) | [Helico](#) | [Open Athena projects](#) | [AstaBench](#)

9.1 / A four-week biomedical extension

Week 9: Specify

Choose one task, primary outcome and realistic comparator. Write the data license/provenance record, exclusion rules, split strategy and annotation protocol. Define what would count as failure.

Week 10: Establish a baseline

Run a frozen baseline on development data. Inspect enough examples to form an error taxonomy. For scored human judgments, write a rubric and check agreement on a shared subset.

Week 11: Test one change

Change only one major factor: data mixture, retrieval method, training objective or inference strategy. Keep a compute and data log. Repeat affordable runs with different seeds when stochasticity could change the conclusion.

Week 12: Evaluate and report

Use the untouched test set only after choices are settled. Report denominators and uncertainty where appropriate, domain gains and general regressions, failure examples, limitations and reproduction instructions.

A minimal evaluation table

Dimension	What to record
Task performance	Task-specific metric, sample count, baseline and uncertainty
Unsupported claims	Rate under a defined annotation protocol and examples
Omissions	Missing required facts or fields; specify the denominator
Calibration / abstention	Confidence quality where measurable; coverage versus error
Robustness	Performance by predefined subgroup, source or perturbation
Efficiency	Latency, memory, compute and data volume
Reproducibility	Commit, model/data versions, seeds, commands and environment

This is a research planning framework. Criteria should be selected for the particular use case; a benchmark score alone is not a clinical deployment decision.

10 / Apprenticeship and research habits

Target 1 - Marin / Open Athena

- Best if you want pretraining, JAX/Levanter-style systems work, transparent experiments, data mixtures, scaling, and AI-for-science adjacency.
- Good first contributions: reproduce a small experiment, add a well-scoped biomedical/scientific evaluation, improve documentation after verifying it yourself, or investigate a clearly bounded open issue.

Target 2 - MarFold / Helico / MarDNA

- Best if you want foundation-model engineering embedded directly in structural biology or genomics.
- Bring domain value: biomedical evaluation design, clinically meaningful failure analysis, cancer-relevant datasets with clean licensing, variant interpretation tasks, or experiment documentation.

Target 3 - Ai2 OLMo / Tulu / Asta

- Best if you want a mature fully open PyTorch ecosystem with released data, checkpoints, recipes, evaluation, and post-training work.
- OLMo-core teaches the training stack; Tulu teaches post-training; Asta connects models to scientific workflows.

A contribution ladder

- Read one completed experiment discussion and write a one-page reconstruction of its hypothesis, controls and failure modes.
- Run a small example and identify a concrete mismatch, missing evaluation or documentation gap.
- Prepare a minimal artifact that lets a maintainer reproduce your observation.
- Propose one change and its success criterion. Wait for project guidance before expensive or broad work.
- Contribute a scoped evaluation, verified documentation improvement, data audit or experiment report.

Study links: [Marin](#) | [OLMo-core](#) | [MarFold](#) | [Helico](#) | [MarDNA](#) | [Tulu](#) | [AstaBench](#)

A concise contribution note

I reproduced [experiment/component] at [commit] with [configuration]. I expected [result] and observed [result]. The attached commands and logs isolate [difference]. I propose [small next step], evaluated by [metric or check].

11 / Tutor use and competency checklist

- Ask Codex to explain one function or tensor path, then predict the behavior yourself before running it.
- Ask it to trace one batch through data loader > embeddings > attention > logits > loss > backward.
- Ask it to generate diagnostic questions, not the final assignment implementation.
- For Marin/OLMo, ask: locate the exact implementation of optimizer, attention, dataloader, checkpointing, distributed strategy, eval loop, and serving/export path.
- Maintain a learning log: concept; where it appears in code; experiment where it matters; failure mode; your reproduction.

Competency checklist

- I can explain a forward pass and backward pass without hand-waving.
- I can write causal self-attention from scratch and explain every tensor dimension.
- I understand why KV caching speeds autoregressive decoding and why it creates a memory bottleneck.
- I can distinguish prefill from decode and latency from throughput.
- I can explain BF16, gradient accumulation, FlashAttention, and basic distributed-training strategies.
- I understand data filtering, deduplication, contamination, and mixture design.
- I can describe pretraining, continued pretraining, SFT, preference optimization, RLHF/RLVR, and inference-time scaling.
- I can read an OLMo/Marin experiment config and identify the major training decisions.
- I can reproduce a small experiment before proposing a new one.
- I can design medical evaluation beyond MedQA accuracy.
- I can enter an open-source research project with a concrete, testable contribution.

Final practical rule

For every 2-3 hours of video, spend at least 2 hours touching code or running an experiment. The goal is not to finish courses. The goal is to become able to open a Marin/OLMo/biology-model repository, trace a training run end-to-end, modify one assumption, predict what should happen, run it, and explain the result.

For assessed coursework, follow the course policy. CS336 allows conceptual and low-level programming help but prohibits directly using an LLM to solve assignment problems. Use predictions, explanations and diagnostic questions to keep the learning yours.

Study links: [CS336 policy and course page](#)

11.1 / Templates for repeatable progress

Learning log

Date | concept | explanation in my words | exact code path | prediction | observation | remaining question | next experiment.

Experiment note

Question and hypothesis. Baseline. One changed factor. Dataset/splits and license.

Model/config/commit. Hardware and precision. Seeds and budget. Metrics and figures. Expected versus observed result. Alternative explanations. Next discriminating experiment.

Course progress

Resource and lesson. Started/completed date. Exercise attempted. Evidence saved. Checkpoint passed. Topic to revisit. Do not count a watched lecture as a passed practical checkpoint.

Weekly review

What can I now implement or explain unaided? Which assumption did an experiment change? What failed? What is the smallest next step? What should I stop reading until I have a reason to return?

Compute and storage discipline

Start scalar and tensor exercises on the hardware already available. Use a small corpus and tiny model for correctness. Run GPU-specific experiments on supported hardware only after correctness is established. Estimate runtime from a short pilot; set a spending limit before a long run. Avoid downloading full datasets and checkpoints merely to browse a project.

Stream lectures in a browser and keep compact notes/configs locally. Store large datasets and checkpoints on a suitable remote environment when needed. No additional app is required to read this PDF or follow its course links.

12 / Source notes and resource index

Study links: [Primary Google Doc](#) | [Linked conversation](#)

The complete Google Doc was retrieved, including embedded hyperlink destinations. The conversation entries available through the app confirmed that PDF was intended and that links should appear inside learning sequences and weekly plans. Prior downloadable attachments from that conversation were not used as source files.

Checked 30 September 2026: Stanford CS336 course page and inference video; Karpathy Zero to Hero; Lambert course; Open Athena projects; MarinFold and Helico repositories; Berkeley Advanced LLM Agents and Deep RL; Ai2 training, Tulu and AstaBench entry points; and MarinDNA repository references.

The CMU pages returned little readable text, so their detailed syllabus claims are retained from the Google Doc rather than independently reverified. The Berkeley Understanding LLMs page could not be retrieved during this check. Existing video and repository links are preserved; this document does not claim every video or lesson was watched or every repository executed.

Updates from verification: CS336 currently identifies Spring 2026; the dedicated inference link is CS336 lecture 10; Lambert lists additional lectures 11-13; CellOLMo was not verified as a named official project. Schedules and effort estimates are editorial additions. No change has been made to the source Google Doc.

Pinned source links

- Karpathy Zero to Hero - <https://karpathy.ai/zero-to-hero.html>
- 3Blue1Brown Attention - <https://www.youtube.com/watch?v=eMlx5fFNoYc>
- Karpathy micrograd - <https://www.youtube.com/watch?v=VMj-3S1tku0>
- Karpathy Build GPT - <https://www.youtube.com/watch?v=kCc8FmEb1nY>
- Karpathy GPT-2 reproduction - <https://www.youtube.com/watch?v=l8pRSuU81PU>
- Stanford CS336 - <https://cs336.stanford.edu/>
- CMU Language Model Inference - <https://www.phontron.com/class/lminference-fall2025/>
- CMU Advanced NLP - <https://www.phontron.com/class/anlp2024/>
- Nathan Lambert course - <https://rlhfbook.com/course>
- Ai2 docs - <https://docs.allenai.org/>
- OLMo-core - <https://github.com/allenai/OLMo-core>
- Berkeley Understanding LLMs - https://rdi.berkeley.edu/understanding_llms/s24
- Berkeley Advanced LLM Agents - <https://rdi.berkeley.edu/adv-llm-agents/sp25>
- Berkeley Deep RL - <https://rail.eecs.berkeley.edu/deeprlcourse/>
- Marin - <https://github.com/marin-community/marin>
- Open Athena - <https://openathena.ai/>

- Open Athena projects - <https://openathena.ai/projects/>
- MarinFold - <https://github.com/Open-Athena/MarinFold>
- Helico - <https://github.com/Open-Athena/helico>
- mumwelt - <https://github.com/Open-Athena/mumwelt>

Additional direct navigation

[3Blue1Brown neural-network playlist](#)

https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi

[CS336 2026 recordings](#)

<https://www.youtube.com/watch?v=JuoVZkPBik&list=PLoROMvodv4rMqXOcazWaTUUHhq-yembLCV>

[CS336 inference video](#)

<https://www.youtube.com/watch?v=EfM546A79aM>

[RLHF book](#)

<https://rlhfbook.com/>

[MarinDNA](#)

<https://github.com/Open-Athena/marin-dna>

[Tulu](#)

<https://allenai.org/tulu>

[AstaBench](#)

<https://allenai.org/asta/bench>

[AstaBench code](#)

<https://github.com/allenai/asta-bench>

Companion web page: <https://inventcures.github.io/llm-learning-path/>